

Optimizador de Imágenes de Seguridad Nocturnas

Materia: Técnicas de Procesamiento Digital de Imágenes

Año: 2026

Profesor: Juan Ignacio Bonini

Integrantes: Michelle Palma y Carolina Cappato

1. Problema

Las cámaras de seguridad en estacionamientos nocturnos capturan imágenes con bajo contraste y poca visibilidad, dificultando la identificación de vehículos y personas. Los principales desafíos son:

- Bajo contraste: los vehículos oscuros se confunden con el fondo
- Ruido del sensor: al compensar la falta de luz, los sensores introducen ruido visual del tipo sal y pimienta o gaussiano
- Iluminación irregular: zonas muy iluminadas conviven con zonas completamente oscuras en la misma imagen

2. Enfoque

Decisión de acotar el caso de uso

Inicialmente el proyecto intentó funcionar con cualquier tipo de imagen nocturna. Durante las pruebas comprobamos que los parámetros calibrados para una escena dejaban de funcionar al cambiar de imagen, porque la iluminación y el tamaño de los objetos variaban demasiado entre escenas con vegetación, calles y jardines.

Por eso decidimos acotar el proyecto a un caso de uso real y fijo: el monitoreo nocturno de estacionamientos con cámara fija. Al trabajar siempre con el mismo tipo de entorno, pudimos calibrar el pipeline con precisión y obtener resultados consistentes.

Dataset utilizado

Las imágenes de prueba provienen del dataset **NDISPark** (Night and Day Instance Segmented Park), publicado en Zenodo por Ciampi et al. (2022). Es un conjunto de imágenes de estacionamientos reales tomadas de día y de noche con siete cámaras distintas, bajo distintas condiciones climáticas y ángulos de visión.

Fuente: <https://zenodo.org/records/6560823>

3. Técnicas utilizadas

3.1 Conversión a escala de grises

La imagen original en color (formato BGR) se convierte a escala de grises porque las técnicas de mejora de contraste y detección de bordes operan sobre un solo canal de información. El color no aporta información relevante para detectar objetos en condiciones nocturnas.

3.2 CLAHE

CLAHE (Contrast Limited Adaptive Histogram Equalization) divide la imagen en sectores y mejora el contraste de cada uno por separado. A diferencia de una simple subida de brillo global, CLAHE respeta las diferencias locales de iluminación, evitando que las zonas ya brillantes se saturen y pierdan detalle.

Esta técnica es especialmente útil en escenas nocturnas con iluminación irregular, como estacionamientos donde algunos vehículos reflejan la luz artificial mientras otros permanecen en zonas oscuras.

Parámetros utilizados:

- clipLimit = 2.0: limita la amplificación del contraste para evitar saturación
- tileGridSize = (8,8): divide la imagen en 64 sectores para procesamiento localizado

3.3 Suavizado gaussiano

Antes de detectar bordes se aplica un filtro gaussiano para reducir el ruido introducido por el sensor nocturno. Sin este paso, el detector de bordes interpreta el granulado de la imagen como bordes reales, generando contornos falsos.

Parámetro utilizado:

- blur_kernel = 5: tamaño del área de suavizado

3.4 Detección de bordes con Canny

Canny detecta cambios bruscos de intensidad en la imagen que corresponden a los bordes de los objetos. Funciona en dos pasos: primero identifica todos los posibles bordes, luego descarta los que no superan los umbrales definidos.

Parámetros utilizados:

- umbral_inferior = 30: cambios de brillo menores a este valor se ignoran
- umbral_superior = 150: cambios mayores a este valor se consideran bordes seguros

3.5 Cierre morfológico

Después de Canny se aplica un cierre morfológico para unir bordes fragmentados

que pertenecen al mismo objeto. Sin esta etapa, un mismo vehículo puede aparecer dividido en múltiples contornos independientes.

Se utilizó un kernel de `(2,2)` en lugar del valor estándar `(3,3)` porque con el kernel más grande los bordes de vehículos cercanos se fusionaban formando rectángulos gigantes que abarcaban varios autos a la vez.

3.6 Detección de regiones brillantes

Como estrategia complementaria a Canny, se detectan zonas con alta luminosidad mediante umbralización binaria. Esta etapa permite capturar vehículos claros cuyos bordes son difusos y no son bien detectados por Canny.

Parámetro utilizado:

- umbral_brillo = 160: píxeles más brillantes que este valor se consideran regiones de interés

3.7 Filtrado de contornos

No todos los contornos detectados representan vehículos reales. Se aplican los siguientes filtros:

- Área mínima (`6000 px<sup>2</sup>`): descarta ruido y objetos muy pequeños
- Área máxima (`70000 px<sup>2</sup>`): descarta regiones demasiado grandes que corresponden al fondo
- Proporción ancho/alto (`0.3 a 4.0`): descarta formas extremas que no corresponden a vehículos
- Posición vertical: descarta contornos en el borde inferior de la imagen

4. Implementación

Estructura del proyecto

proyecto_final/

```
|— entrada/      — imágenes de prueba del dataset NDISPark
|— procesamiento/
|  |— __init__.py
|  |— configuracion.py — parámetros centralizados y ajuste automático
|  |— mejora.py     — clase MejoradorDeImagen
|  |— deteccion.py  — clase DetectorDeObjetos
|— pruebas/
|  |— debug/        — imágenes intermedias de cada etapa
|— salida/         — imágenes procesadas resultantes
|— main.py         — coordinador del pipeline
```

Separación de responsabilidades

El Core de procesamiento está completamente desacoplado de la capa de entrada y salida:

- `mejora.py` contiene la clase “MejoradorDeImagen” con la lógica de mejora de visibilidad
- `deteccion.py` contiene la clase `DetectorDeObjetos` con la lógica de detección de objetos
- `configuracion.py` centraliza todos los parámetros del pipeline
- `main.py` actúa como coordinador: carga la imagen, llama al Core y guarda el resultado

Ajuste automático de parámetros

La aplicación analiza cada imagen antes de procesarla y ajusta algunos parámetros automáticamente:

- Imagen oscura (brillo promedio < 50): aumenta `clahe\_clip\_limit`

- Imagen ruidosa (desviación estándar > 60): aumenta `area\_minima` y `canny\_inferior`
- Imagen sobreexpuesta (brillo promedio > 180): aumenta `canny\_superior`

5. Pruebas y resultados

Proceso de Calibración

Con el objetivo de encontrar la configuración óptima para el procesamiento de imágenes de estacionamiento, se realizaron múltiples pruebas modificando de forma iterativa los valores de los parámetros.

Registro de Pruebas y Resultados

Nº de Prueba	Cambio Aplicado	Resultado Observado
1	Parámetros por defecto: area_minima: 500	Demasiadas detecciones. El modelo marcaba ruido pequeño de la imagen.
2	Incremento de área y brillo: area_minima: 3000 umbral_brillo: 200	Se observó una mejora, pero los rectángulos grandes tendían a agrupar varios autos en una sola detección.
3	Ajuste de kernel: Reducción de kernel morfológico de (3,3) a (2,2)	Se lograron rectángulos más precisos y delimitados por cada objeto de forma individual.
4	Ajuste de límites de área (Configuración Final): area_minima: 6000 area_maxima: 70000	Resultado final equilibrado. Filtrado óptimo de ruido y delimitación correcta de los vehículos.

Resultado final

El sistema detecta correctamente vehículos blancos y claros, furgonetas y personas en el estacionamiento. Los vehículos oscuros representan el caso más difícil por su bajo contraste con el fondo nocturno.

6. Limitaciones conocidas

- Los vehículos oscuros tienen poco contraste con el fondo nocturno, lo que dificulta su detección tanto por bordes como por brillo
- Los parámetros están calibrados para este tipo de escena específica y pueden no funcionar bien en otros entornos
- Cuando dos vehículos están muy próximos, sus contornos pueden fusionarse en un solo rectángulo

7. Mejoras futuras

- Implementar un modelo de detección de objetos entrenado específicamente para reconocer vehículos, lo que permitiría detectar autos oscuros independientemente del contraste.
- Agregar un contador de vehículos para medir la disponibilidad de espacios en tiempo real.
- Implementar zonas de exclusión para ignorar áreas fijas de la imagen que siempre generan falsos positivos.

Anexo - Uso de Inteligencia Artificial

Cómo usamos la IA

Usamos la IA principalmente para entender conceptos que no teníamos claros y para resolver problemas que surgían durante el desarrollo.

Consultas conceptuales

Consultamos qué era CLAHE y por qué era mejor que una simple subida de brillo global. Entendimos que la diferencia está en que CLAHE trabaja por zonas, preservando los detalles locales de la imagen en vez de afectar toda la imagen por igual.

También consultamos cómo funciona el algoritmo de Canny y por qué necesita dos umbrales en vez de uno. Entendimos que el umbral inferior descarta cambios de brillo pequeños que son ruido, y el superior confirma los bordes reales.

Decisiones técnicas

Consultamos por qué los resultados eran inconsistentes al cambiar de imagen. La IA nos ayudó a entender que un detector genérico sin contexto fijo no es viable, y que la solución era acotar el proyecto a un caso de uso específico. A partir de esa conclusión decidimos enfocarnos en estacionamientos nocturnos.

También consultamos cómo afecta el kernel morfológico a la detección. Entendimos que un kernel muy grande fusiona objetos cercanos en un solo contorno, por lo que lo redujimos de `(3,3)` a `(2,2)` para obtener rectángulos más precisos.